Quiz 2: MCP in Practice

Test your understanding of building, configuring, and deploying MCP systems. Choose the best answer.

Question 1: FastMCP Server Building

- [x] Use FastMCP decorators (@mcp.tool()) to define functions as MCP tools

Correct! Correct! FastMCP infers types from function signatures and generates schemas automatically.

- [ ] Write raw JSON-RPC messages for each tool
- [ ] Gradio is the only way to build MCP servers
- [ ] MCP servers must be written in JavaScript

Submit **You got all the answers!**

Question 2: Type Hints and Docstrings

- [x] Type hints and docstrings are essential for MCP tool schemas

Correct! Exactly! Type hints generate schemas, docstrings provide descriptions. Always include both.

- [ ] Type hints are optional; MCP infers types from usage
- [ ] Only Gradio MCP requires type hints
- [ ] Docstrings are just for human readers

Submit You got all the answers!

Question 3: Tools vs. Resources vs. Prompts

- [x] Tools: actions | Resources: read-only data | Prompts: instructions

Correct! Perfect! Each serves a different purpose in agent workflows.

- [ ] Tools and Resources are interchangeable
- [ ] Resources are for UI only, not for agents
- [ ] Prompts are the same as Tools

Submit You got all the answers!

Question 4: Gradio MCP Integration

- [x] Use demo.launch(mcp_server=True) to enable MCP automatically

Correct! Correct! One parameter enables both web UI and MCP endpoint.

- [ ] Gradio and MCP are separate; you need both libraries
- [ ] You need to manually implement the MCP protocol in Gradio
- [ ] Gradio MCP only works with simple functions

Submit You got all the answers!

Question 5: Configuring Agents to Use MCP Servers

- [x] Configure servers via CLI commands (like claude mcp add) or config files, depending on the agent

Correct! Correct! Claude Code uses CLI commands; other agents use config files. Remote Streamable HTTP servers can often be added without restarts.

- [ ] Agents automatically discover all available MCP

- [ ] You need a special GUI tool to configure MCP servers
- [ ] Each agent needs its own custom MCP SDK integration

Submit **You got all the answers!**

Question 6: Deploying to Hugging Face Spaces

- [x] Create a Space, upload app.py with mcp_server=True, add requirements.txt

Correct! Perfect! Spaces auto-builds and exposes the MCP endpoint.

- [ ] Spaces doesn't support MCP yet
- [ ] You need to configure network ports and manage server processes
- [ ] MCP servers on Spaces only work with private access

Submit **You got all the answers!**

Question 7: Transport Mechanisms

- [x] Use stdio for local servers, Streamable HTTP for remote deployment

Correct! Exactly! Stdio is fast for subprocess communication. Streamable HTTP is the current standard for remote MCP connections.

- [ ] Streamable HTTP is always better because it's 'network-native'
- [ ] Once deployed, you must switch from stdio to Streamable HTTP
- [ ] Transport is internal to MCP; agents don't see the difference

Submit **You got all the answers!**

Question 8: MCP Security

- [x] Use environment variables for secrets; never hardcode them in config

Correct! Excellent! Env vars keep secrets out of ve

- [ ] Hardcode API tokens in config if it's 'private'
- [ ] Use relative paths in server configuration
- [ ] There's no security consideration for MCP configuration

Submit **You got all the answers!**

Summary

How many did you get right?

- **7-8 correct**: You're ready to build and deploy real MCP systems!
- **5-6 correct**: Good understanding. Review configuration and deployment sections.
- **3-4 correct**: Review the building and integration sections.
- **0-2 correct**: Go back through the unit and review the core concepts.

Key Takeaways

- **FastMCP** is the simplest way to build servers with decorators
- **Type hints and docstrings** are essential for MCP tool schemas
- **Gradio with** `mcp_server=True` creates UI + MCP automatically
- **Configure servers in agent config files** (or use MCPClient in code)
- **Use stdio locally, Streamable HTTP for remote deployment**
- **Always use environment variables** for secrets
- **Deploy to Hugging Face Spaces** for easy sharing

You've mastered Unit 2! You can now build, deploy, and configure MCP servers for any agent.

</> Update on GitHub

← Hands-On: Build and Deploy an MCP Server Unit 3: Plugins →